



Sheet 3, starting from 18th of March, 2026, due 8th of June, 2026

Introduction and General Comments: You will need to use the original images of the dataset, which can be found here:

<https://zenodo.org/record/1324999>

Note that the ScriptNet denotes the test set. Please use the binarized version of both sets, i.e., “icdar17-historicalwi-training-binarized.zip” denotes the training set and “ScriptNet-HistoricalWI-2017-binarized.zip” the test set. We will provide you with label files for the training and test set (icdar17_labels_train.txt and icdar17_labels_test.txt) which contain for each image a class index (i.e., the ground truth writer label).

Note: in case you copy or run out of quota in your home folder: use your personal ciptmp-folder, i.e. /proj/ciptmp/<yourlogin>, it offers you 20GB (no backup). If the folder for your login doesn’t exist, create it on your own. (If you cannot see the ciptmp folder, you have to enter it first — it is a shared network drive). See also: <https://www.cip.informatik.uni-erlangen.de/documentation/overview.en.html>

- We also provide in StudOn a code skeleton (‘skeleton.py’), which we recommend to use.
- Test your algorithms with different parameters and try to get a deeper understanding regarding their behaviors.

Exercise 3.1: Writer Identification

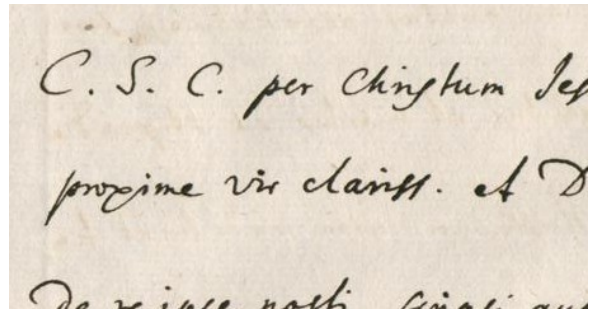
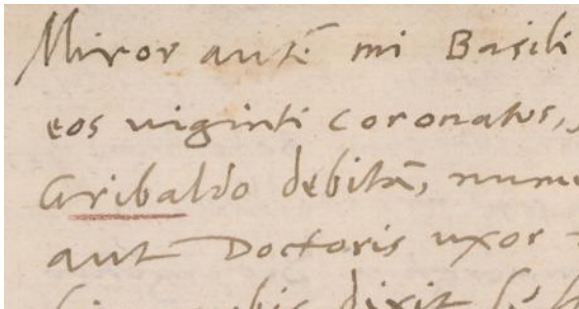


Abbildung 1: Image excerpts of the ICDAR2017 Historical WI Dataset: 21-IMG_MAX_966414 (left), 21-IMG_MAX_1006529 (right).

In this exercise, we will implement a small writer identification system. It is going to be evaluated using the ICDAR17 Historical WI benchmark dataset. The dataset is divided into a training set (a.k.a. experiment set) and a test set (a.k.a. benchmark set). Please download the dataset and the code skeleton.

The *bag of (visual) words* model for image recognition/retrieval is going to be used, where each image is represented by a global vector representation, which is formed by encoding the local feature descriptors. The global descriptor is eventually used for the classification/retrieval.

We will use VLAD encoding, which later gets extended by means of *generalized max pooling*. Given T local descriptors $\mathcal{X} = \{\mathbf{x}_t \in \mathbb{R}^D | t = 1, \dots, T\}$ and a dictionary (or codebook) $\mathcal{D} = \{\boldsymbol{\mu}_k \in \mathbb{R}^D | k =$

$1, \dots, K\}$, the VLAD encoding (i.e. VLAD embedding + sum pooling, see lecture) for one cluster is: $\xi_k(\mathcal{X}) = \sum_{t=1}^T \alpha_k(\mathbf{x}_t)(\mathbf{x}_t - \boldsymbol{\mu}_k)$. The overall encoding is: $\boldsymbol{\xi}(\mathcal{X}) = (\xi_1^T, \dots, \xi_K^T)^T \in \mathbb{R}^{DK}$.

a) Extract your own features

We want now to compare the pre-computed descriptors with traditional SIFT descriptors. Therefore, create a `computeDescs` method, which takes a filename as input, loads the filename and computes SIFT descriptors at SIFT keypoints. As improvements, we will modify (1) the keypoints: set all keypoints' angles to 0; (2) SIFT descriptors: apply a Hellinger normalization (this is in essence similar to a power normalization). However, because the SIFT descriptors are already ℓ^2 -normalized, we have to first apply an ℓ^1 normalization and afterwards take the sign square root of each element of the vector (no ℓ^2 normalization afterwards required).

b) Codebook generation

For computing the residuals $(\mathbf{x}_t - \boldsymbol{\mu}_k)$, we need a codebook to generate the dictionary $\mathcal{D}$. To do so, a representative number of descriptors needs to be selected. Therefore, use a random selection of about 500 000 descriptors from the *training* features.

Compute a codebook (a. k. a. vocabulary/background model) with the help of *k*-means ($k = 100$). For this operation you can use sklearn's `MiniBatchKMeans`.

c) VLAD encoding

With the help of the computed dictionary, we can now compute the residuals (see above). They are multiplied with the indicator/association function $\alpha_k(\mathbf{x}_i)$, which returns 1 for the nearest cluster center and 0 otherwise. Therefore, you can use `cv2.BFMatcher` and `knnMatch`.

Suggestion: compute first the full association matrix (i.e. a vector for each descriptor. Each element of the vector represents the association to a cluster. That means, set all elements of the vector to 0 except the nearest cluster center). This can then be used to obtain for each cluster only those descriptors that have this cluster assigned. Then, compute the residuals with these descriptors and aggregate them to one global descriptor.

Compute the pairwise distances between all encodings and evaluate the performance in terms of mean average precision (mAP).

d) VLAD normalization

In order to improve the representational power of VLAD encoding, we should account for visual burstiness, i.e. frequent local descriptors that dominate the overall similarity. This can be achieved by using *power normalization*: $\xi'_i = \text{sign}(\xi_i) \sqrt{|\xi_i|}$, $\forall \{\xi_i\} \in \boldsymbol{\xi}$, where `sign` is +1 if $\xi_i \geq 0$ and -1 otherwise. Don't forget to ℓ^2 normalize, i. e., divide by the ℓ^2 norm, the representation again afterwards. Compute the pairwise distances between all normalized encodings and evaluate the performance in terms of mean average precision (mAP) and compare with the result before.

e) Exemplar classification

The last step of the pipeline is the proposed exemplar classification. Therefore, you have to compute an individual SVM for each global representation (encoding) of the *test* set using this encoding as positive and all the encodings of the training set as negatives. (sklearn's `LinearSVC`), a good *C* value is 1000 (actually this value should be cross-validated with a different validation set), also set `class_weight` to `balanced`. Afterwards, ℓ^2 normalize the weight vector (`coef_` parameter of the SVM) and use it as your new global descriptor.

Compute the pairwise distances between all newly created descriptors and evaluate the performance in terms of mean average precision (mAP) and compare with the result before.

10 ECTS individual exercise

You are free to do whatever you like to potentially improve your results – of course it is fine if the results eventually won't have improved. I just want to see that you spend some efforts in improving the results. You are free to get help by an LLM (we cannot stop the future).

Here just some ideas:

- In class, you learned about Generalized Max-Pooling, you could try this out.
- Whitening often helps to improve the results, e.g., by PCA whitening.
- Use a CNN/ViT backbone to generate features instead of using hand-crafted ones, in the past most people tried self-supervised ones (e.g., DINOv2 features). You could also try out metric-learning using the writers of the training set.
- Try out other VLAD variations, e.g., NetVLAD, GhostVLAD or SuperVLAD.
- Re-ranking significantly helps to improve retrieval results.
- Improve somehow the input. For example improve the binarization or just do a one-sided binarization (i.e., don't set the ink to 0 and only set the background to 1. Sidenote: for training a Neural Network, it is actually better to invert the images so that the script is white and the background 0).
- You could also think of using GraphNN or reinforcement learning to improve results further.
- I guess there are plenty of other tricks/things you could try out...

Submission Please submit your code `exercise3.py` and a PDF or TXT file in which you report the results for each of the experiments. I'd say that at least two different reasonable things should be tried out to improve the results.